# musiccloud Resolver-Flow

musiccloud backend

Version 1.0.0+022e409b · 2026-04-30

Architektur und Datenfluss durch den Resolver

## Contents


**1 Was dieses Dokument beschreibt** **2**

**2 Wie eine Anfrage hereinkommt** **2**

2.1 Der authentifizierte Endpunkt . . . 2

2.2 Der unauthentifizierte Endpunkt . . . 3

2.3 Wie ein Token ausgestellt wird . . . 3

2.4 Verwandte Endpunkte . . . 4

**3 Was der Dispatcher entscheidet** **4**

**4 Der Weg einer URL** **5**

4.1 Aufräumen der URL . . . 5

4.2 Die drei Cache-Stufen . . . 6

4.3 Die Quelle bestimmen . . . 6

4.4 Metadaten abrufen . . . 8

**5 Der Weg einer Textsuche** **10**

5.1 Der erste Durchgang . . . 10

5.2 Mit ausgewähltem Vorschlag zurückkommen . . . 12

**6 Genre-Discovery** **14**

6.1 Browse-Modus . . . 14

6.2 Suchmodus . . . 16

**7 Dienstübergreifende Suche** **18**

**8 Persistenz** **20**

8.1 Der kanonische Schreibvorgang . . . 20

8.2 Begleitvorgänge und Vorschau-Erneuerung . . . 22

9 Cache-Strategie an einer Stelle 24

10 Konfidenz-Schwellwerte 24

11 Hinweise für den Betrieb 25

12 Eine neue Verzweigung oder Strategie hinzufügen 25

# 1 Was dieses Dokument beschreibt

Wer einen Spotify-Link in musiccloud einfügt, möchte am Ende eine Share-Seite erhalten, die denselben Titel auch auf Apple Music, Tidal, Deezer, YouTube und einer Reihe weiterer Dienste öffnet. Genau diese Übersetzung leistet der Resolver. Das vorliegende Dokument beschreibt seinen Aufbau und den Datenfluss durch seine einzelnen Schichten.

Der Resolver verarbeitet fünf verschiedene Arten von Eingaben. Jede dieser Eingaben durchläuft einen leicht abweichenden Pfad durch das System. Sobald ein Ergebnis vorliegt, enden jedoch alle Pfade am selben Punkt: in der Datenbank, beim Share-Link und in der Antwort, die der Nutzer zu sehen bekommt. Was zwischen Eingabe und Ergebnis geschieht, ist der eigentlich interessante Teil. Hier fangen Caches wiederkehrende Anfragen ab, vermitteln Adapter zwischen den Streaming-Diensten, und sorgt eine Reihe von Notlösungen dafür, dass das System auch dann noch funktioniert, wenn einer der Dienste vorübergehend ausfällt.

# 2 Wie eine Anfrage hereinkommt

Der Resolver ist über zwei HTTP-Endpunkte erreichbar. Beide verwenden denselben Dispatcher und dieselben nachgelagerten Abläufe; sie unterscheiden sich darin, wer sie aufrufen darf, wie oft das geschehen darf und in welcher Form sie mit dem Aufrufer kommunizieren.

## 2.1 Der authentifizierte Endpunkt

`POST /api/v1/resolve` ist der vollwertige Einstiegspunkt. Jede Anfrage muss sich ausweisen, entweder mit einem `X-API-Key`-Header für interne Aufrufer oder mit einem `Authorization: Bearer <jwt>`-Header für externe Clients, die ein OAuth-Token besitzen. Zusätzlich greift eine Anfrage-Begrenzung pro Client-IP, denn ein gestohlener Schlüssel ist nach wie vor ein Schlüssel.

Der Anfragekörper enthält genau eines von zwei Feldern: entweder `query` mit der URL oder dem Suchbegriff, den der Nutzer eingegeben hat, oder `selectedCandidate`, wenn der Client eine zuvor begonnene Disambiguierung abschließt.

**Table 1:** Anfragekörper für `POST /api/v1/resolve`.

| Feld | Typ | Verwendung | Bedeutung |
|---|---|---|---|
| `query` | string | erste Anfrage | Eingabe des Nutzers, höchstens 500 Zeichen. |
| `selectedCandidate` | string | Folgeanfrage | Kennung eines zuvor angebotenen Treffers, höchstens 200 Zeichen. |

Welche Antwort der Aufrufer erhält, hängt vom Ergebnis ab. Findet der Resolver einen eindeutigen Treffer, antwortet er mit der Erfolgsmeldung, die alle Metadaten, die dienstübergreifenden Links und den Share-Kurzlink enthält. Bei einer Textsuche, deren bester Treffer nicht sicher genug ist, antwortet er stattdessen mit einer Liste von Vorschlägen. Der Client zeigt diese Liste dem Nutzer, nimmt dessen Auswahl entgegen und schickt sie als Folgeanfrage zurück. Bei einer Genre-Anfrage liefert der Resolver eine Trefferliste zur Anzeige aus, ohne in die Datenbank zu schreiben; das geschieht erst, wenn der Nutzer einen der Treffer anklickt.

Im Fehlerfall trägt die Antwort einen Fehlercode, der einem HTTP-Status zugeordnet ist: `400` bei fehlerhafter Eingabe, `401` bei ungültigen Zugangsdaten, `404` bei einer URL, zu der keine Quelle gefunden werden konnte, `408` bei einer Zeitüberschreitung im Backend, `429` bei Erreichen der Anfragegrenze, `500` bei einem unerwarteten Fehler und `503`, wenn ein vom Resolver benötigter Dienst nicht konfiguriert ist.

## 2.2 Der unauthentifizierte Endpunkt

`GET /api/v1/resolve` richtet sich an Werkzeuge, die nicht ohne Weiteres ein Bearer-Token mitführen können: Apple Shortcuts, einzeilige curl-Aufrufe, Bookmarklets und ähnliche Skript-Integrationen. Da hier keine Authentifizierung greift, übernimmt die Anfrage-Begrenzung pro IP allein den Schutz vor Missbrauch. Sie ist deshalb enger gesetzt als am authentifizierten Endpunkt.

**Table 2:** Parameter für `GET /api/v1/resolve`.

| Parameter | Typ | Pflicht | Bedeutung |
|---|---|---|---|
| `query` | string | ja | URL oder Suchbegriff, höchstens 500 Zeichen. |
| `format` | string | nein | `json` (Standard) liefert die vollständige Antwort. `text` liefert ausschließlich den Share-Kurzlink als reinen Text, etwa zur Weiterverarbeitung in einer Pipe. |

Eine Textsuche, die zu mehreren Vorschlägen führt, kann dieser Endpunkt nicht beantworten, weil eine zustandslose Einzelanfrage keinen zweiten Schritt erlaubt. In diesem Fall antwortet er mit `400`. Die übrigen Fehlercodes entsprechen denen des authentifizierten Endpunkts; nur `401` entfällt, da es keine Authentifizierung gibt, die scheitern könnte.

Ein typischer Aufruf mit curl sieht so aus:

```
curl -G 'https://api.musiccloud.example/api/v1/resolve' \
  --data-urlencode 'query=https://open.spotify.com/track/...' \
  --data-urlencode 'format=text'
```

Das Ergebnis ist der Share-Kurzlink, bereit zum Einfügen in eine Nachricht oder zur Weitergabe an einen weiteren Befehl.

## 2.3 Wie ein Token ausgestellt wird

Bevor ein externer Client den authentifizierten Endpunkt nutzen kann, tauscht er seine Zugangsdaten gegen ein JWT ein. Das geschieht unter `POST /api/auth/token` nach dem Client-Credentials-Verfahren von OAuth 2.0.

**Table 3:** Felder der Token-Anfrage.

| Feld | Typ | Bedeutung |
|---|---|---|
| `client_id` | string | Bei musiccloud registrierte Client-Kennung. |
| `client_secret` | string | Das zugehörige Geheimnis. |
| `grant_type` | string | Muss exakt `client_credentials` lauten. |

Die Antwort enthält drei Felder: `access_token`, das JWT, das in nachfolgenden Anfragen als `Authorization: Beare` mitgeschickt wird, `token_type` mit dem festen Wert `Bearer` und `expires_in` mit der Gültigkeitsdauer in Sekunden, derzeit eine Stunde. Läuft das Token ab, fordert der Client mit denselben Zugangsdaten einfach ein neues an. Einen Refresh-Token-Mechanismus gibt es bewusst nicht.

### 2.4 Verwandte Endpunkte

Zwei weitere Endpunkte gehören zur Resolver-Erfahrung, liegen aber außerhalb des Themas dieses Dokuments:

- `GET /api/v1/share/{shortId}` liefert die Share-Seite zu einem zuvor aufgelösten Eintrag, gefunden über seine Kurz-ID.
- `GET /api/v1/share/{shortId}/preview` fordert eine abgelaufene oder fehlende Vorschau-URL bei Bedarf neu an. Der Audio-Player im Frontend ruft diesen Endpunkt auf, sobald er feststellt, dass eine zwischengespeicherte Vorschau nicht mehr abspielbar ist.

## 3 Was der Dispatcher entscheidet

Beide Endpunkte münden in denselben ersten Schritt: festzustellen, welche Form die Eingabe hat. Fünf Ergebnisse sind möglich, und der Dispatcher prüft sie der Reihe nach.

Enthält der Anfragekörper das Feld `selectedCandidate` statt `query`, ist dies der zweite Schritt einer laufenden Disambiguierung. Die Kandidatenkennung enthält genug Information, um den Faden dort wieder aufzunehmen, wo die erste Anfrage geendet hat. Der Dispatcher reicht direkt an den entsprechenden Handler weiter; dieser Pfad existiert nur am authentifizierten Endpunkt.

Lautet die Eingabe wörtlich `genre: ?`, liefert der Dispatcher das vorbereitete Genre-Raster aus, ohne irgendetwas aufzulösen. Dieses Raster ist es, das auf der Discovery-Seite erscheint; ein Klick darauf führt anschließend zu einem regulären Resolve-Aufruf.

Beginnt die Eingabe mit dem Schlüsselwort `genre:`, so handelt es sich um eine strukturierte Genre-Suche, etwa `genre: jazz, count: 20`. Der Dispatcher gibt sie an die Genre-Schicht weiter, die die Syntax auswertet und Anfragen an das Discovery-Backend stellt.

Lässt sich die Eingabe als URL lesen, klassifiziert der Dispatcher sie weiter: Album-URLs gehen an den Album-Resolver, Artist-URLs an den Artist-Resolver, alles übrige an den Track-Resolver. Die drei Resolver teilen den größten Teil ihrer Mechanik und unterscheiden sich vor allem darin, mit welchen Kennungen sie arbeiten.

Greift keine dieser Regeln, behandelt der Resolver die Eingabe als Textsuche. Der authentifizierte Endpunkt kann darauf entweder mit einem einzelnen Treffer oder mit einer Vorschlagsliste antworten; der unauthentifizierte erlaubt nur den eindeutigen Fall.

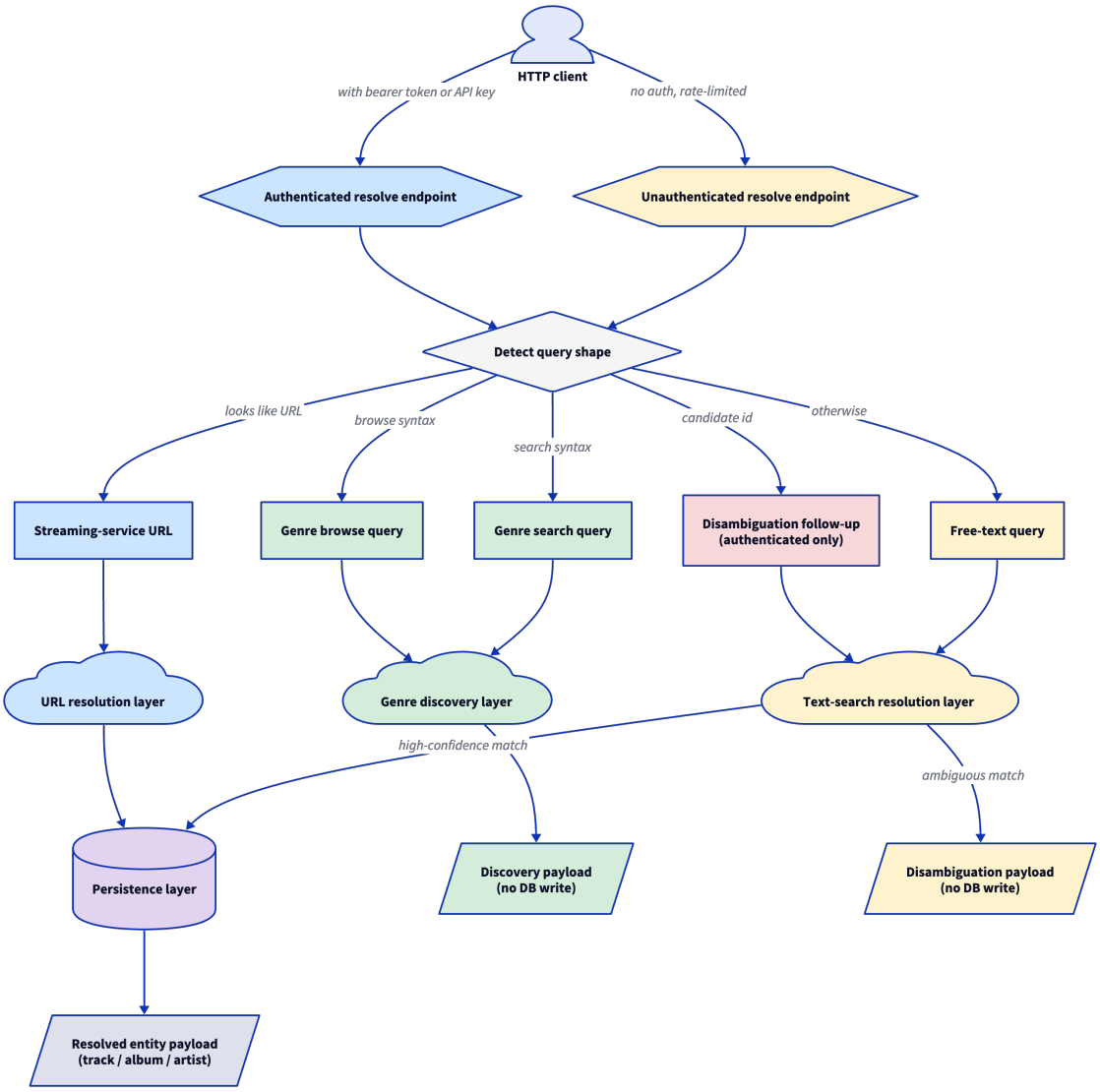


Figure 1: Die fünf möglichen Ausgänge des Dispatchers.

# 4 Der Weg einer URL

URLs sind die häufigste Eingabeform und nehmen den aufwendigsten Pfad. Album- und Artist-URLs zweigen früh in eigene Resolver ab. Die wesentliche Mechanik liegt jedoch im Track-Pfad, weshalb dieser Abschnitt diesem folgt.

## 4.1 Aufräumen der URL

Der erste Schritt besteht darin, die URL zu bereinigen. URLs aus dem freien Netz sind selten sauber. Sie tragen Spuren von Marketing-Kampagnen, Social-Media-Buttons und Analytics-Werkzeugen in Form zusätzlicher Parameter. Zwei Aufrufe, die auf denselben Spotify-Track verweisen, sich aber im Wert von `utm_source` unterscheiden, würden ohne diese Bereinigung als zwei unterschiedliche Cache-Einträge geführt. Erst das Entfernen der Parameter macht den Cache nützlich.

Manche Anbieter verbergen die eigentliche URL hinter einer Weiterleitung, etwa `link.deezer.com/s/...` oder `on.soundcloud.com/...`. Der Resolver folgt solchen Weiterleitungen mit einer HTTP-HEAD-Anfrage, um die kanonische URL zu erhalten, und bereinigt anschließend erneut. Häufig fügt das Ziel der Weiterleitung wieder eigene Parameter ein, sodass eine einmalige Bereinigung nicht ausreicht.

### 4.2 Die drei Cache-Stufen

Bevor der Resolver einen externen Dienst kontaktiert, prüft er drei Caches in fester Reihenfolge.

Die erste Suche erfolgt anhand der bereinigten URL. Wurde derselbe Track schon einmal über genau diese URL aufgelöst, ist der Datenbankeintrag noch gültig. Seit der Umstellung der Cache-Architektur laufen kanonische Einträge nicht mehr ab; sie werden nur ergänzt, sobald neue Adapter zur Verfügung stehen.

Die zweite Suche nutzt den ursprünglichen Kurzlink als Alias. Wurde eine URL aus einem Kurzlink aufgelöst, speichert der Resolver den Originallink als zusätzlichen Schlüssel. Der nächste Aufruf desselben Kurzlinks trifft auf den Cache, ohne die Weiterleitung erneut zu bezahlen.

Die dritte Suche stützt sich auf den ISRC[1], die Industriekennung der Aufnahme. Sie wird erst nach dem Abruf der Metadaten beim Quelldienst geprüft. Auf dieselbe Aufnahme können URLs verschiedener Dienste zeigen; über den ISRC erkennt der Resolver diesen Fall und verwendet den vorhandenen Eintrag erneut.

### 4.3 Die Quelle bestimmen

Bringt keine der drei Cache-Stufen einen Treffer, muss der Resolver herausfinden, zu welchem Dienst die URL gehört. Er befragt jedes installierte Plugin, auch jene, die der Betreiber gerade abgeschaltet hat. Das wirkt zunächst widersinnig, ist aber beabsichtigt: Wer einen Tidal-Link einfügt, erfährt so im Fehlerfall, dass Tidal abgeschaltet ist, statt eine allgemeine Meldung wie "kein Musik-Link" zu erhalten. Welche Plugins aktiv sind, entscheidet der Betreiber; welcher Link eingefügt wird, der Nutzer. Die Fehlermeldung soll diesen Umstand widerspiegeln.

Erkennt kein Plugin die URL, antwortet der Resolver mit der allgemeinen Meldung "kein Musik-Link". Erkennt sie ein Plugin, das aber abgeschaltet ist, gibt er die genauere Meldung "Dienst nicht aktiv" zurück.

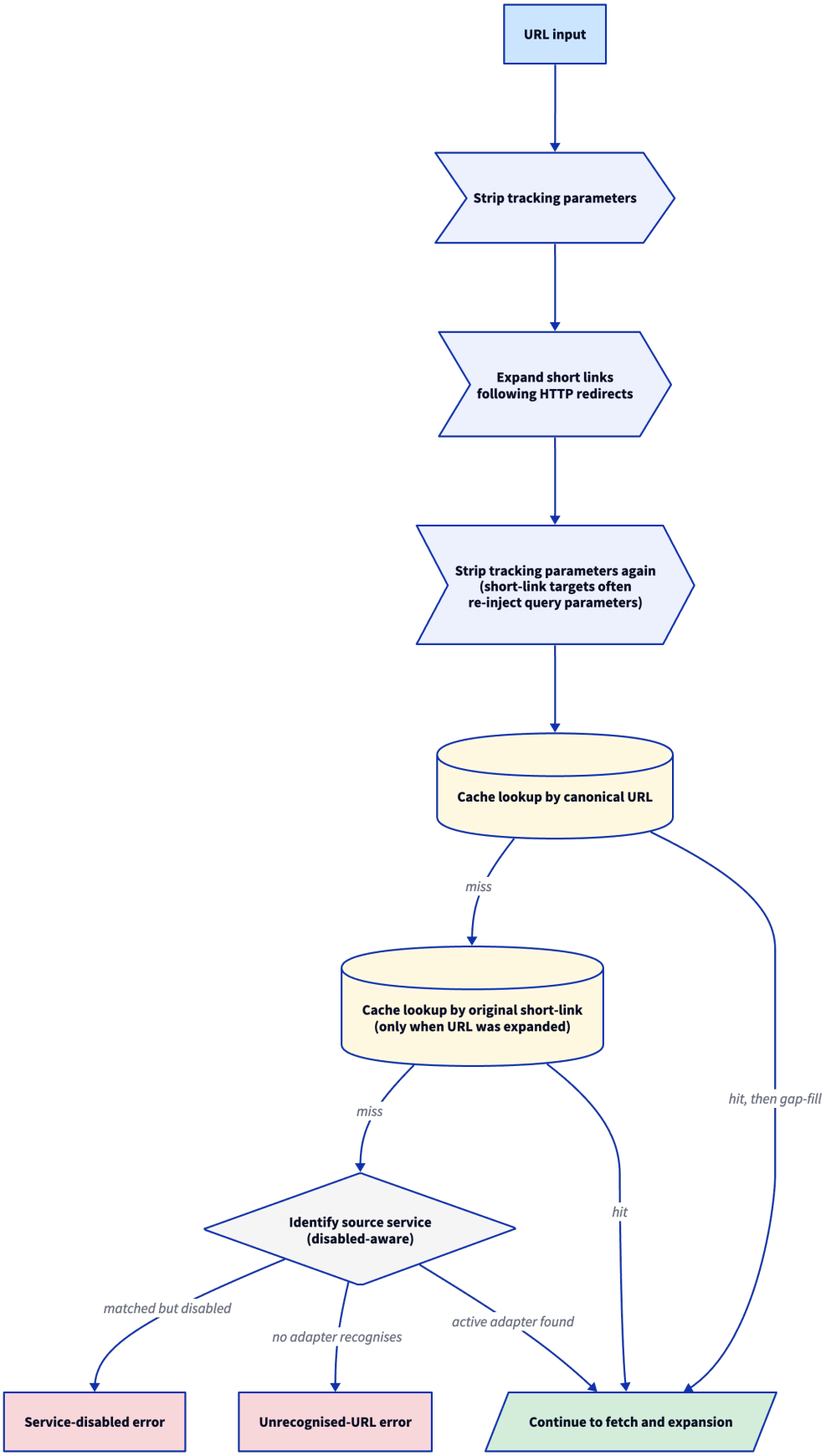


Figure 2: URL-Pfad: Cache-Stufen und Bestimmung der Quelle.

### 4.4 Metadaten abrufen

Sobald die Quelle feststeht, ermittelt der zuständige Adapter die Track-Kennung aus der URL und fragt beim Quelldienst die Metadaten an: Titel, Interpret, Album, ISRC, Dauer, Cover und die URL der Hörprobe.

Die Adapter teilen sich in zwei Gruppen. Die erste Gruppe verfügt über API-Zugänge: Spotify, Apple Music, Deezer, Tidal und die meisten weiteren Anbieter. Sie liefern strukturierte Antworten. Die zweite Gruppe arbeitet ohne Zugang und liest die öffentliche Webseite des Dienstes aus. Wird ein solcher Adapter ohne konfigurierten Schlüssel aufgerufen, fordert der Resolver die Seite mit einer User-Agent-Kennung an, wie sie ein Bot eines sozialen Netzwerks verwenden würde, und liest die Open-Graph-Metaangaben aus dem zurückgelieferten HTML aus. Dieser Umweg ist nötig, weil viele moderne Webseiten nur dann servergerendert antworten, wenn sie einen bekannten Crawler erkennen; einem normalen Browser liefern sie ein leeres JavaScript-Gerüst aus, das die Inhalte erst nachträglich nachlädt.

Liegen die Metadaten der Quelle vor, läuft die dienstübergreifende Suche parallel gegen alle weiteren registrierten Adapter; der Abschnitt weiter unten beschreibt sie im Detail. Vor dem Bauen der Antwort folgen zwei Aufräumschritte.

Der erste Schritt betrifft das Cover. Manche Dienste liefern in ihrer Track-API kein Coverbild. Hat die Quelle keines, ist aber ein ISRC bekannt, holt sich der Resolver ein Cover aus den Treffern der anderen Dienste. Spotify und Apple Music stehen am Anfang dieser Reihenfolge, weil beide gleichbleibend hochwertige Cover ausliefern.

Der zweite Schritt betrifft die Hörprobe. Vorschauen gibt es in zwei Geschmacksrichtungen: dauerhaft gültige CDN-URLs auf der einen Seite und signierte URLs mit kurzer Lebenszeit auf der anderen. Eine signierte URL, die heute funktioniert, kann morgen mit `403` antworten. Der Resolver wählt deshalb stets die stabilste verfügbare Vorschau aus. In der Praxis bedeutet das, dass eine Deezer-Vorschau eine Spotify-Vorschau in jedem Fall verdrängt, selbst wenn Spotify die ursprüngliche Quelle war.

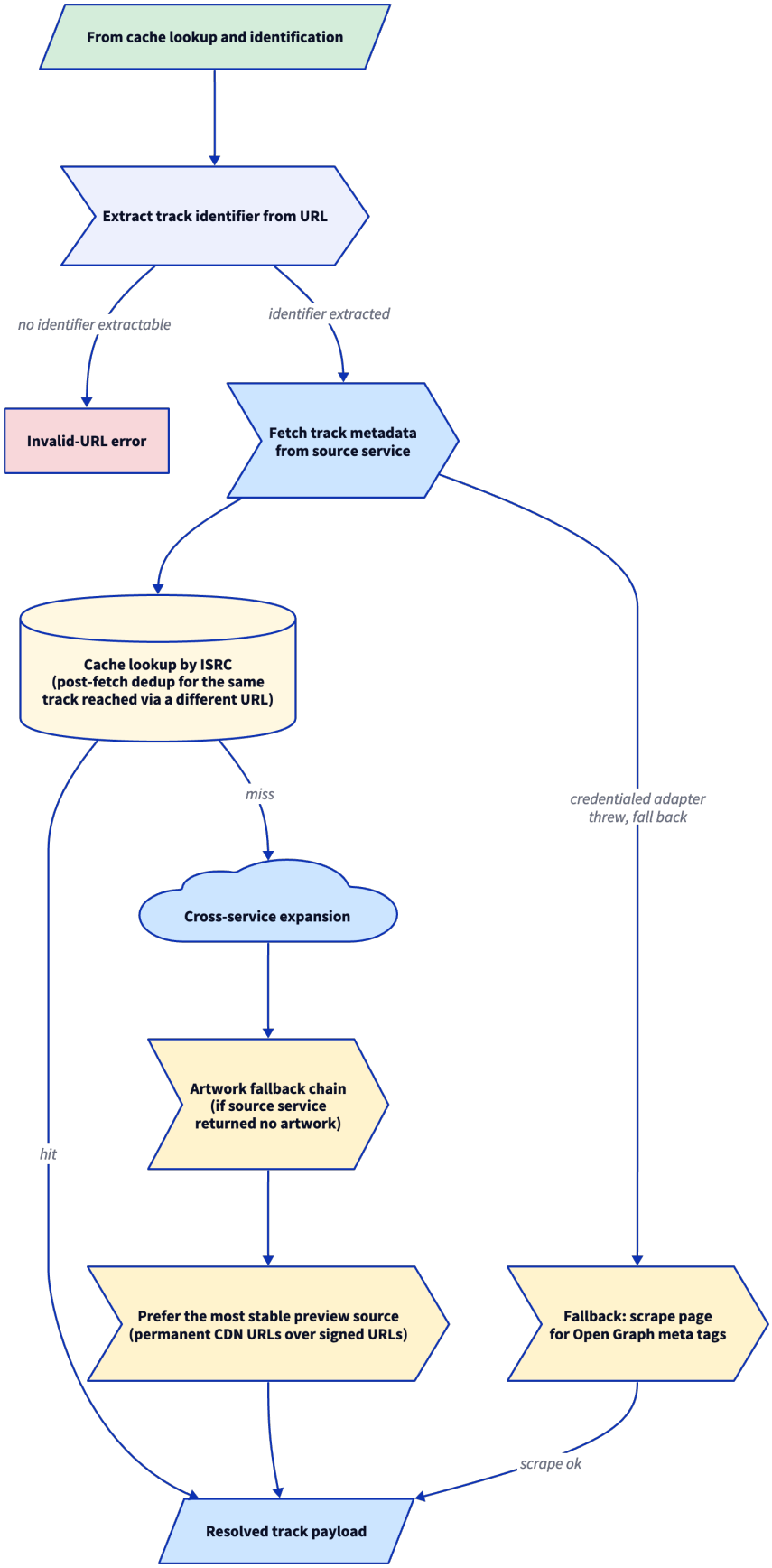


Figure 3: URL-Pfad: Metadatenabruf und dienstübergreifende Suche.

# 5 Der Weg einer Textsuche

Die Textsuche fängt alles auf, was weder eine URL noch eine Genre-Anfrage ist. Wer etwa `daft punk around the w` eingibt, erwartet einen Track als Ergebnis.

An dieser Stelle unterscheiden sich die beiden Endpunkte. Der authentifizierte Endpunkt kann mit dem Client einen Dialog führen: Liefert die Suche keinen klaren Treffer, schickt er eine Liste von Vorschlägen zurück, aus der der Client einen auswählt. Der unauthentifizierte Endpunkt kennt keinen Dialog. Er antwortet nur dann, wenn das Ergebnis eindeutig ist, und gibt sonst `400` zurück.

## 5.1 Der erste Durchgang

Der Resolver durchläuft die aktiven Adapter in einer festgelegten Reihenfolge. Ist Spotify konfiguriert, kommt es zuerst zum Zug, denn die Relevanzwertung der Spotify-Suche liefert die zuverlässigste Grundlage. Die übrigen Adapter dienen als Auffangnetz.

Auch hier teilen sich die Adapter in zwei Gruppen, diesmal nach dem Verhalten ihrer Such-API. Manche liefern nur das beste Einzelergebnis mit einem Konfidenzwert; andere geben eine sortierte Liste mit Werten je Treffer zurück. Erst die zweite Variante macht Disambiguierung überhaupt möglich.

Liefert der Adapter nur ein Einzelergebnis, kann der Resolver es entweder direkt übernehmen, sofern es überzeugend genug aussieht, oder zum nächsten Adapter weitergehen. Liefert er eine Liste, prüft der Resolver den vordersten Treffer. Liegt sein Konfidenzwert über der Schwelle für eine direkte Auswahl, nimmt der Resolver ihn ohne Rückfrage an. Andernfalls schickt er die Liste an den Client.

Drei Schwellwerte regeln dieses Verhalten. Der erste entscheidet, ob der vorderste Treffer die Disambiguierung überspringen darf. Der zweite filtert Treffer aus der Vorschlagsliste, deren Konfidenz zu gering ist, um den Nutzer überhaupt mit ihnen zu behelligen. Der dritte begrenzt die Größe der Liste, damit eine unscharfe Anfrage keine endlos lange Antwort erzeugt.

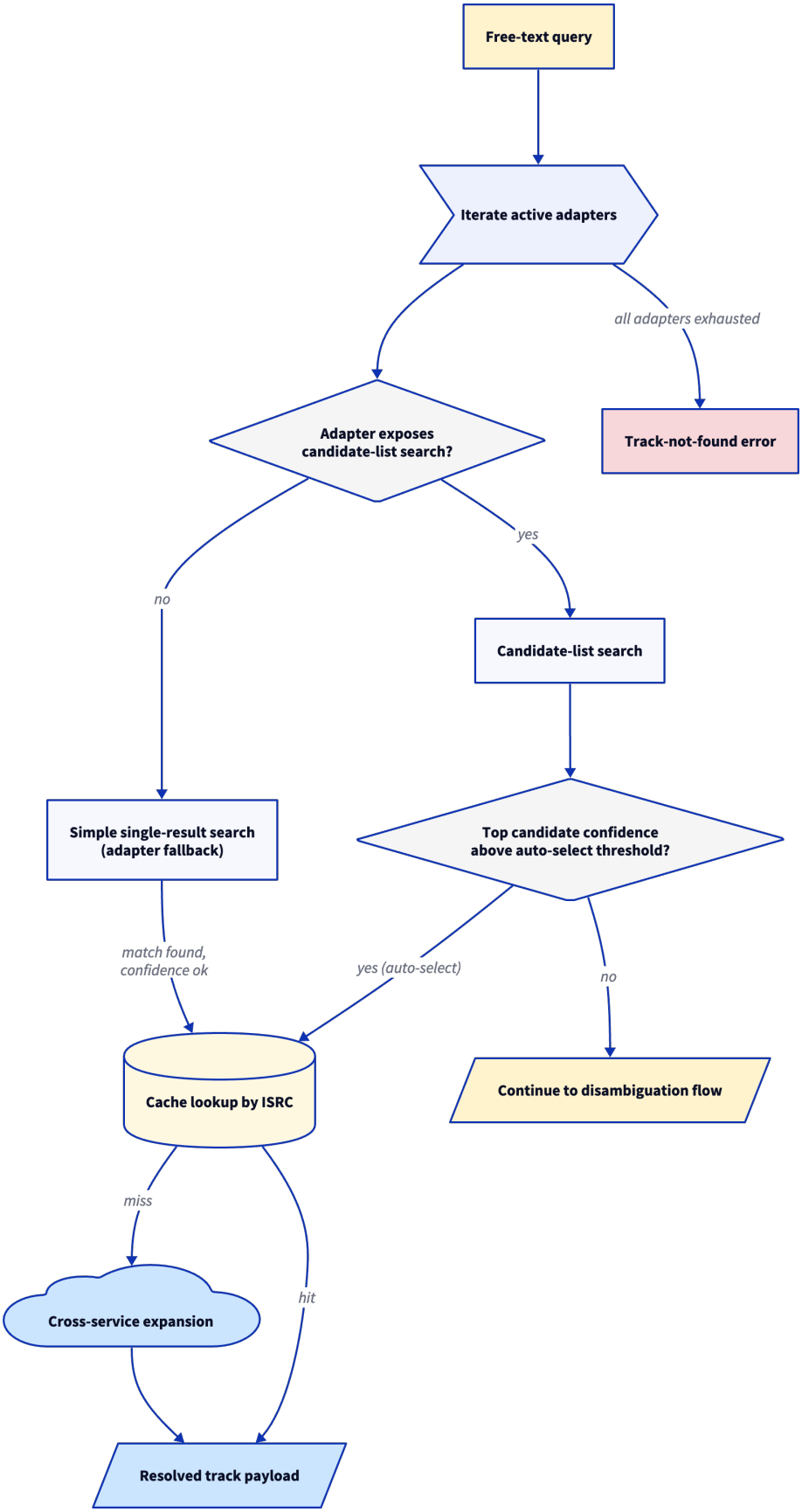


Figure 4: Textsuche, erster Durchgang.

### 5.2 Mit ausgewähltem Vorschlag zurückkommen

Disambiguierung ist eine zweistufige Konversation. Auf die erste Anfrage hin liefert der Resolver die Vorschlagsliste, der Client zeigt sie dem Nutzer, und sobald dieser einen Vorschlag ausgewählt hat, sendet der Client eine zweite Anfrage mit dem Feld `selectedCandidate`. Diese zweite Anfrage nimmt einen anderen Pfad: Der Resolver holt die vollständigen Metadaten des gewählten Treffers vom Quelldienst, prüft den Cache anhand des ISRC und führt anschließend die dienstübergreifende Suche durch, genau wie bei einer URL-Auflösung.

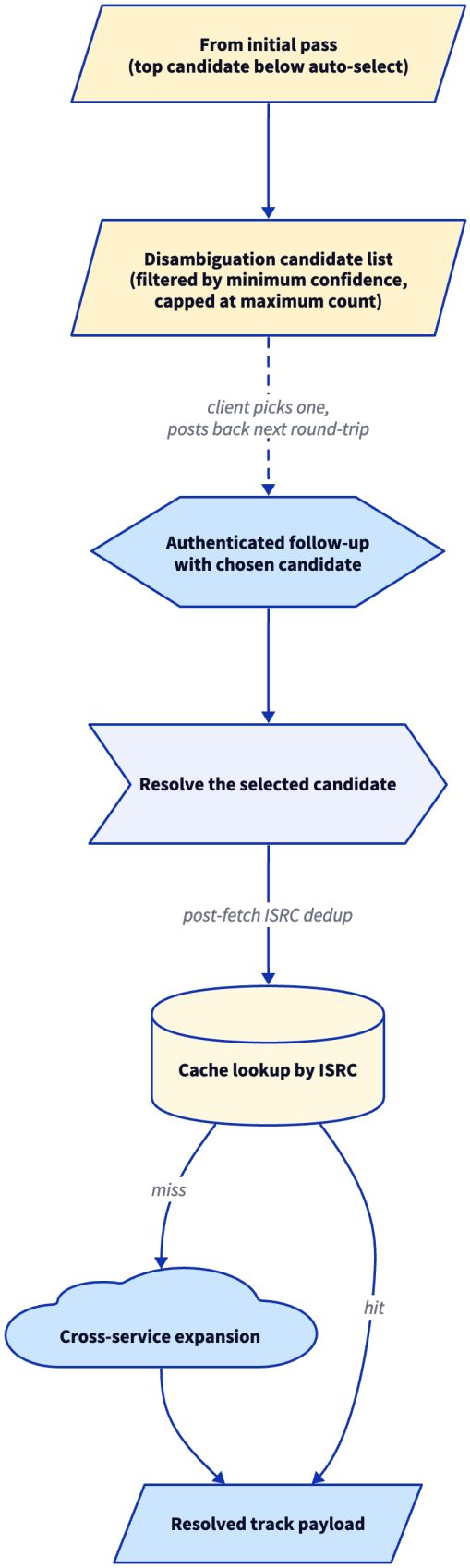


Figure 5: Textsuche, Folgeanfrage zur Disambiguierung.

Bringt kein einziger Adapter ein verwertbares Ergebnis, antwortet der Resolver mit der Meldung "Track nicht gefunden". Das geschieht in der Praxis am häufigsten bei Tippfehlern, bei sehr unbekannten Aufnahmen und bei Eingaben, die der Nutzer gar nicht als Suche nach einem Musiktitel gemeint hat.

# 6 Genre-Discovery

Der Genre-Pfad ist der einzige Teil des Resolvers, der nichts in die kanonische Datenbank schreibt. Er liefert nur eine Auswahl an Vorschlägen aus. Klickt der Nutzer einen davon an, gelangt das Klickziel über den URL- oder Textpfad in den Resolver und wird dort regulär gespeichert.

Das Discovery-Backend stützt sich auf von der Community vergebene Tags und kennt dadurch tausende Subgenres, weit über das hinaus, was die großen Streaming-Kataloge direkt anbieten. Trance, Krautrock, Shoegaze, Bebop, Vaporwave, Dungeon Synth und ähnliches finden sich nur dort. Ohne den passenden API-Schlüssel ist der gesamte Genre-Pfad nicht nutzbar; das ist eine der wenigen Stellen, an denen ein Feature tatsächlich von einem einzelnen externen Dienst abhängt.

## 6.1 Browse-Modus

Schickt der Nutzer wörtlich die Anfrage `genre: ?`, antwortet der Resolver mit dem vorbereiteten Genre-Raster: einigen hundert Tags, gefiltert gegen eine Sperrliste (Länder, Jahrzehnte, Instrumente und persönliche Sammlungsbezeichner sind keine Genres) und um Schreibvarianten bereinigt, sodass dieselbe Genrebezeichnung nicht mehrfach im Raster erscheint. Das gesamte Raster wird für 24 Stunden im Speicher vorgehalten; die aufwendige Vorbereitung läuft also nur einmal pro Backend-Neustart.

Jede Kachel im Raster verweist auf eine relative URL, die auf einen separaten Endpunkt zeigt. Dieser rendert das Coverbild beim ersten Aufruf und legt es im Cache ab; alle nachfolgenden Aufrufe erhalten das fertige Bild.

```
GET /api/v1/resolve?query=genre%3A+%3F
```

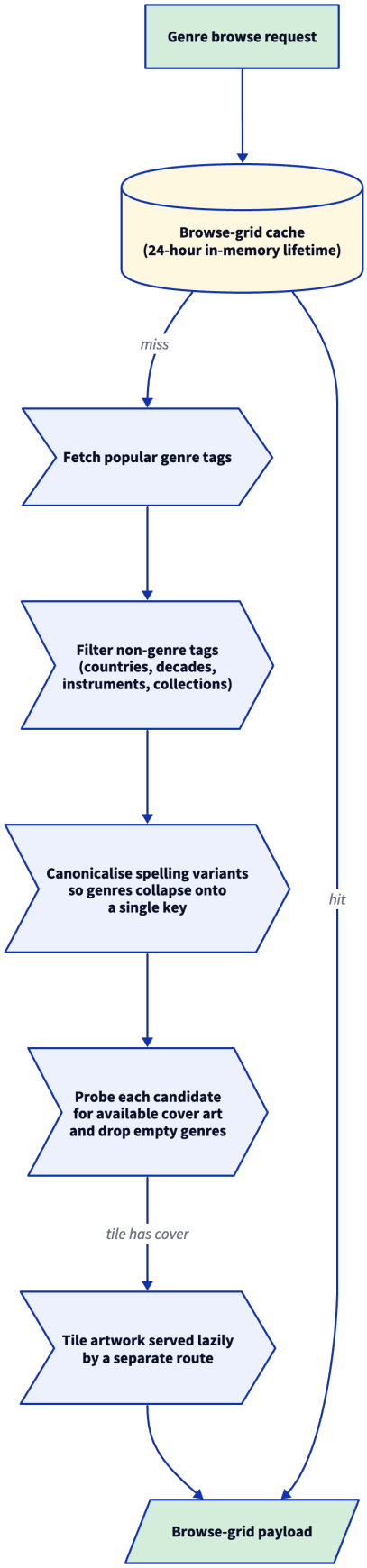


Figure 6: Genre-Discovery, Browse-Modus.

## 6.2 Suchmodus

Beginnt die Eingabe mit dem Schlüsselwort `genre:`, liest der Resolver sie als strukturiertes Formular ein. Die Grammatik ist bewusst einfach gehalten und erlaubt Anfragen wie diese:

```
genre: jazz
genre: rock | metal
genre: hip hop, count: 20
genre: ambient, tracks: 30, vibe: mixed
genre: classical | baroque, albums: 12, artists: 8
```

Folgende Felder stehen zur Verfügung:

**Table 4:** Felder einer Genre-Suche.

| Feld | Bedeutung |
|---|---|
| `genre` | Eines oder mehrere Genres, getrennt durch `\|`. Pflicht. |
| `tracks` | Anzahl gewünschter Tracks. Standard: 10. |
| `albums` | Anzahl gewünschter Alben. |
| `artists` | Anzahl gewünschter Interpreten. |
| `count` | Kurzform, um dieselbe Anzahl für alle drei Kategorien zu setzen. |
| `vibe` | `hot` bevorzugt populäre Treffer (Standard), `mixed` mischt populäre, mittlere und weniger bekannte Treffer. |

Mehrere Genres in einer Anfrage werden als ODER-Bedingung verstanden: `genre: jazz | bebop` liefert Treffer, die unter mindestens einem der beiden Tags geführt werden. Unbekannte Felder oder ungültige Zahlen führen zu einem Fehler; der Parser ignoriert nichts stillschweigend.

Die beiden Werte für `vibe` unterscheiden sich darin, wie aus den Per-Genre-Daten ausgewählt wird. `hot` nimmt die Spitze der Charts, optional gleichmäßig über die obersten Plätze verteilt, damit nicht alle Treffer aus derselben Kalenderwoche stammen. `mixed` zieht eine geschichtete Stichprobe aus oberem, mittlerem und unterem Drittel; das Ergebnis lehnt spürbar in Richtung Entdeckung statt in Richtung Wiedererkennung.

Sobald die Daten vorliegen, ergänzt der Resolver fehlende Cover aus dem dauerhaften Bild-Cache. Bilder, die hier zum ersten Mal auftauchen, werden ebenfalls in den Cache geschrieben, sodass spätere Anfragen nichts mehr nachladen müssen.

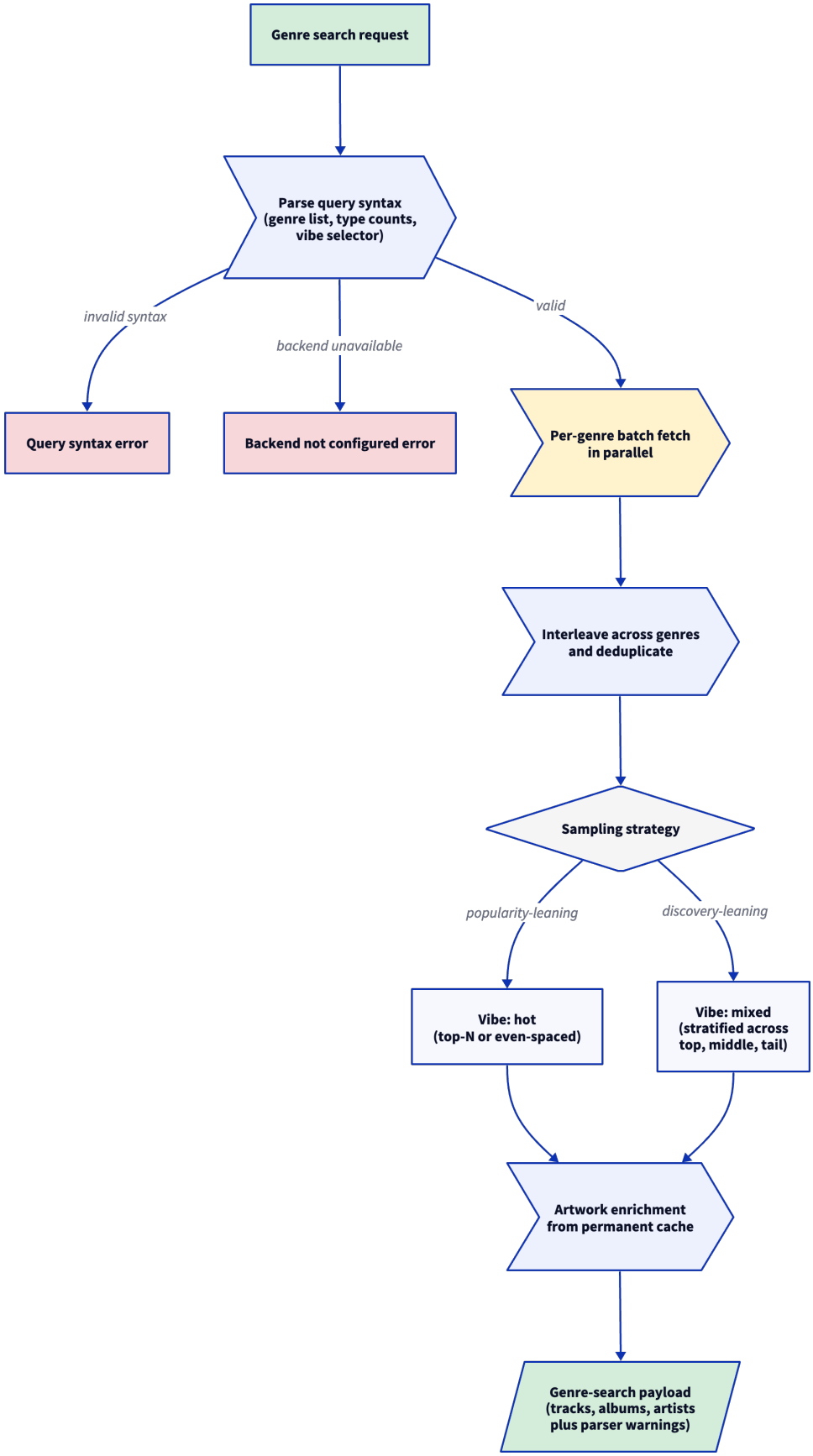


Figure 7: Genre-Discovery, Suchmodus.

# 7 Dienstübergreifende Suche

Dieser Schritt macht den Resolver eigentlich erst nützlich. Der Nutzer hat einen Spotify-Link eingefügt, doch die Share-Seite soll Apple Music, Tidal, Deezer, YouTube und alle übrigen Dienste ebenfalls öffnen. Die dienstübergreifende Suche ist der Mechanismus, mit dem der Resolver denselben Track auf jedem weiteren registrierten Dienst aufspürt.

Sobald die Quelle bekannt ist, läuft jeder andere Adapter parallel gegen sie an. Eine Zeitschranke pro Adapter sorgt dafür, dass ein einzelner blockierender Adapter die gesamte Antwort nicht aufhält; ohne diese Schranke käme erst der allgemeine HTTP-Timeout zum Tragen, der deutlich länger gesetzt ist.

Pro Adapter werden zwei Strategien in absteigender Beweiskraft versucht.

Die stärkere Strategie ist die Suche über die Kennung. Unterstützt der Adapter den ISRC und liegt einer für die Quelle vor, fragt der Adapter im Prinzip: "Habt ihr genau diese Aufnahme?" Das Ergebnis ist eindeutig, mit oder ohne Treffer. Ein Treffer wird mit voller Konfidenz gewertet, denn der ISRC ist die stärkste dienstübergreifende Track-Kennung.

Die schwächere Strategie ist die Suche nach Titel und Interpret. Der Adapter führt die unscharfe Suche durch, die seine API anbietet. Der Konfidenzwert kommt aus dieser API; die Qualität schwankt deutlich von Anbieter zu Anbieter. Spotifys Relevanzwertung ist verlässlich, andere wirken eher wie geschätzte Zahlen.

Treffer unterhalb einer Mindestgüte filtert der Resolver heraus, bevor sie den Nutzer erreichen. Eine Ausnahme bilden synthetische Such-Notlösungen, also Links, die statt direkt auf den Treffer auf eine Suchseite des Dienstes verweisen. Sie bleiben erhalten, damit in der Antwort wenigstens eine Option pro Dienst auftaucht.

Zwei Ableitungen schließen sich an. Aus jedem erfolgreichen YouTube-Treffer leitet der Resolver einen entsprechenden YouTube-Music-Link ab, indem er den Hostnamen austauscht und die Video- oder Kanalkennung übernimmt. Eine separate Suche ist dafür nicht nötig. Findet sich kein direkter YouTube-Treffer, fügt der Resolver einen synthetischen Link auf die YouTube-Suche hinzu, sodass in der Antwort niemals der YouTube-Eintrag ganz fehlt.

Album- und Artist-Suchen folgen einer parallelen Logik mit eigenen Match-Strategien. Bei Alben kommt zuerst die Suche nach UPC[2] zum Einsatz, anschließend die Ableitung des Albums anhand seiner Track-ISRCs (eine Stichprobe der Tracks wird beim Zieldienst nachgeschlagen, und das am häufigsten gemeldete Album gewinnt), und zuletzt die Textsuche. Bei Interpreten nutzt der Resolver eine eigene Konfidenzformel, die exakte Treffer, "The"-Präfix-Varianten, hohe Ähnlichkeit, Teilstring-Treffer und reine Fuzzy-Treffer voneinander unterscheidet.

Sämtliche Schwellwerte stehen in einem zentralen Konstantenmodul. Eine Verhaltensänderung ist somit eine Zahlenänderung in einer Datei und keine Suche nach magischen Konstanten quer durch den Quellcode.

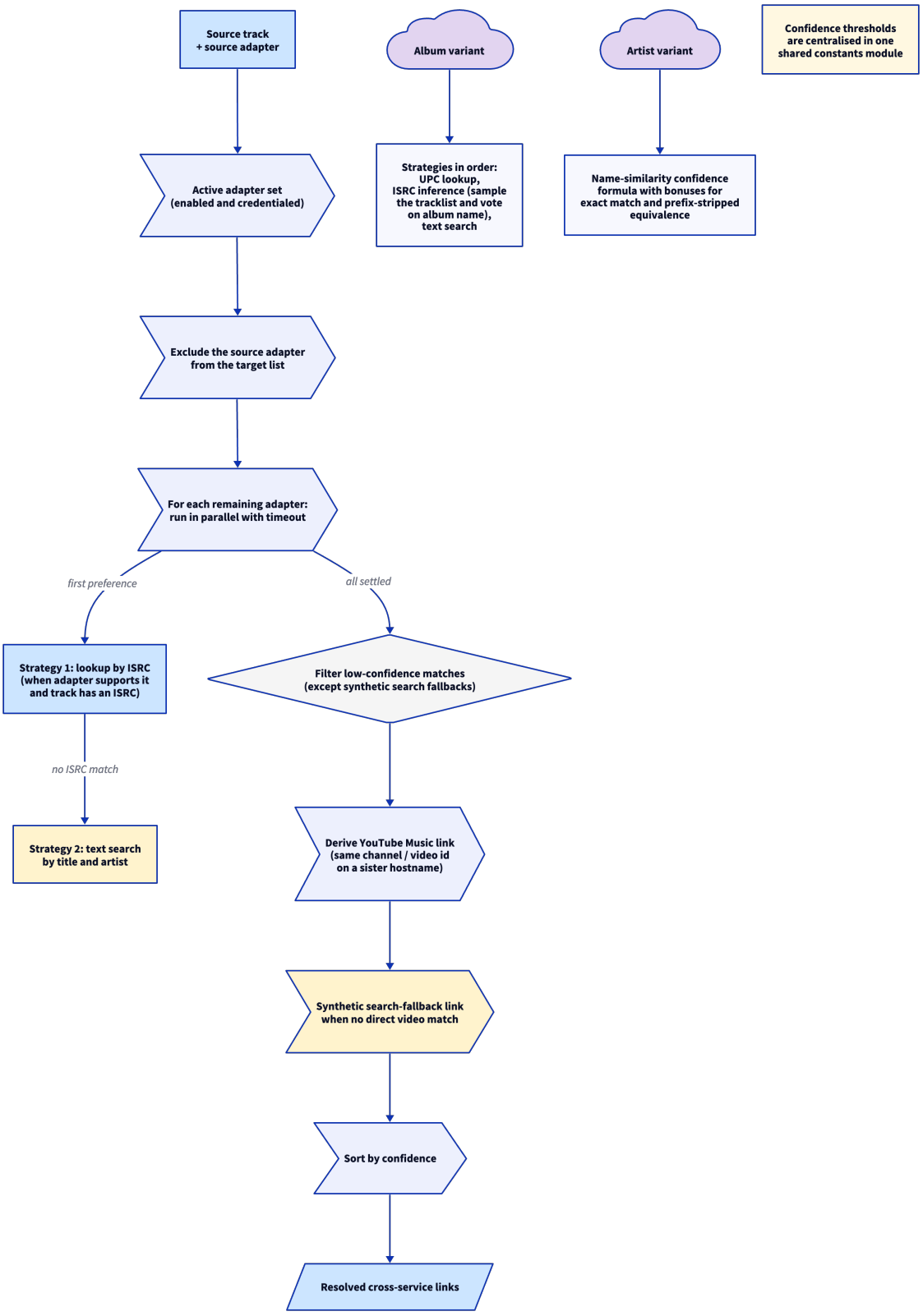


Figure 8: Dienstübergreifende Suche.

# 8 Persistenz

Sobald ein Auflösungsergebnis vorliegt, schreibt die Persistenz-Schicht es in einer festen Reihenfolge in die Datenbank. Diese Schicht teilen sich der Route-Layer und der Crawler-Pfad. Beide Aufrufer behandeln den Begriff "persistiert" dadurch identisch.

## 8.1 Der kanonische Schreibvorgang

Den Anfang macht der kanonische Track-Eintrag mit seinen dienstübergreifenden Links. Der Resolver dedupliziert ihn entweder über den ISRC oder über die Quell-URL. Existiert ein passender Eintrag bereits, werden seine Metadaten aktualisiert; andernfalls entsteht ein neuer Eintrag mit frischen Kennungen. Die dienstübergreifenden Links setzt er per Upsert auf das Paar (Eintrag, Dienst); ein erneuter Aufruf desselben Resolves ist deshalb idempotent. Die Share-Kurzlink-ID wird nur dann angelegt, wenn der Track selbst neu ist; auf diese Weise bleibt der Share-Link über erneute Auflösungen hinweg stabil.

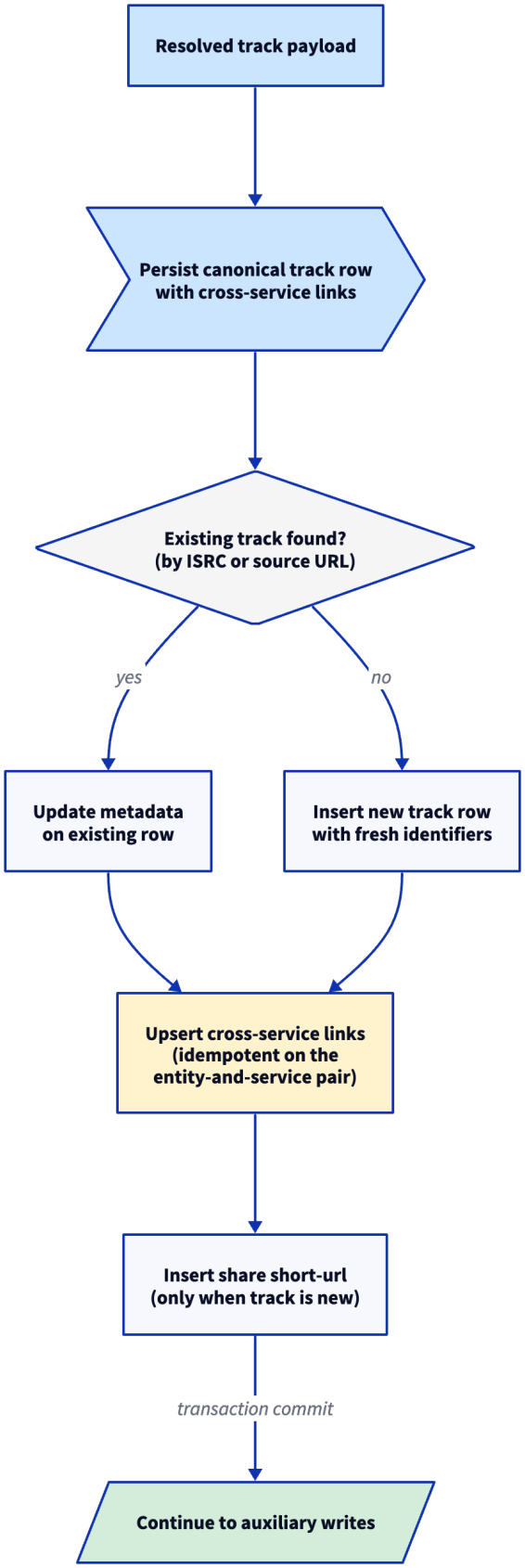


Figure 9: Persistenz, kanonischer Schreibvorgang.

## 8.2 Begleitvorgänge und Vorschau-Erneuerung

Jede beobachtete externe Kennung – ISRC, MBID[3], ISWC[4] und UPC – wird in einer eigenen Aggregationstabelle abgelegt, idempotent über das Paar (Eintrag, Kennungstyp, Wert, Quelldienst) eingefügt. Fehler bei diesem Schritt sind nicht kritisch und werden nur auf Debug-Ebene geloggt; ein gestolperter Aggregations-Schreibvorgang darf eine ansonsten erfolgreiche Auflösung nicht zum Fehler degradieren.

Jeder dienstübergreifende Link, der eine Vorschau-URL geliefert hat, wird per Upsert mit der jeweiligen Ablaufzeit gespeichert. Die Vorschau-URL des Quelldienstes wird auf dieselbe Weise abgelegt.

War die ursprüngliche Eingabe ein Kurzlink, der erst aufgelöst werden musste, speichert der Resolver den Originallink als Alias. Folgeanfragen über denselben Kurzlink treffen so direkt auf den Cache. Schlägt das Schreiben des Alias fehl, geschieht das ohne Konsequenzen für die Antwort; der Alias ist eine reine Optimierung.

Ist die Vorschau-URL fehlend oder abgelaufen und liegt ein ISRC vor, holt der Resolver eine neue URL aus der stabilsten verfügbaren Quelle nach. Aus diesem Grund liefert der Resolver auch nach einem Cache-Treffer noch eine abspielbare Vorschau, selbst wenn die zuvor gespeicherte zwischenzeitlich abgelaufen ist.

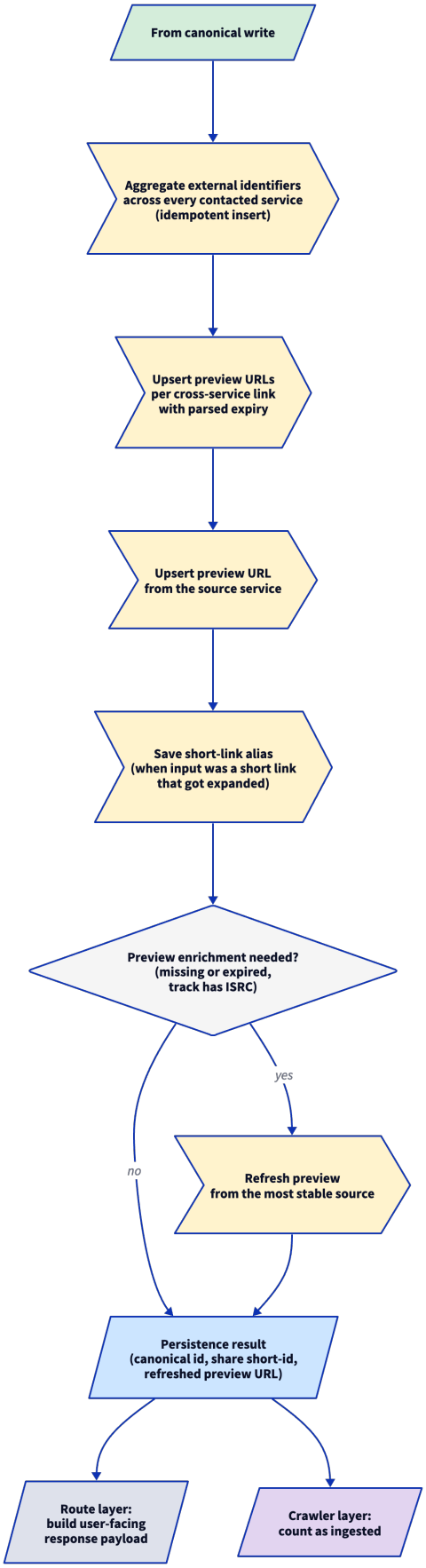


Figure 10: Persistenz, Begleitvorgänge und Vorschau-Erneuerung.

Am Ende gibt die Persistenz-Schicht drei Werte an den Aufrufer zurück: die kanonische Track-Kennung, die Share-Kurzlink-ID und die Vorschau-URL, die er an den Client weitergeben soll. Der Route-Layer baut daraus die nutzersichtbare Antwort; der Crawler verbucht das Ergebnis lediglich als "ingestiert".

Album- und Artist-Persistenz haben eigene Hilfsfunktionen im Route-Layer. Sie sind noch nicht in den geteilten Kern überführt worden, weil es zurzeit keinen zweiten Verbraucher dafür gibt.

## 9 Cache-Strategie an einer Stelle

Jeder lesende Cache-Zugriff verwendet denselben kanonischen Eintrag, den die Persistenz schreibt. Seit der Trennung in statische und dynamische Anteile laufen kanonische Einträge nicht mehr ab. Einmal geschrieben, gilt ein Track-, Album- oder Artist-Eintrag dauerhaft als aktuell. Nur Vorschau-URLs tragen einen Ablaufzeitpunkt und werden bei Bedarf beim Lesen nachgezogen.

Diese Trennung ist es, die dem Resolver erlaubt, neue Dienste nach und nach nachzutragen. Wird heute ein neuer Adapter aktiviert, ergänzt der Resolver schon bei der nächsten Anfrage an einen vorhandenen Track den fehlenden Link; ein Cache-Reset ist dafür nicht nötig. Die Aktualitätsprüfung beschränkt sich auf die Tabelle der Vorschau-URLs: Sie löst eine Erneuerung nur dann aus, wenn keine gültige Vorschau mehr für die jeweilige Quelle existiert. Das Nachholen von Deezer-Vorschauen läuft auch nur in diesem Fall an. Der schnelle Pfad bei einem Cache-Treffer besteht deshalb aus einer einzigen indizierten Suche.

## 10 Konfidenz-Schwellwerte

Sämtliche Schwellwerte liegen in einem zentralen Konstantenmodul und werden vom Einstiegspunkt des Resolvers der Bequemlichkeit halber erneut exportiert. Die maßgeblichen Werte stehen im Modul; die folgende Tabelle fasst zusammen, welche Rolle jeder einzelne Schwellwert spielt.

**Table 5:** Konfidenz-Schwellwerte und ihre Aufgaben.

| Konstante | Was sie regelt | Aufgabe |
|---|---|---|
| `MATCH_MIN_CONFIDENCE` | Ein Adapter-Treffer muss mindestens diesen Wert erreichen, um überhaupt als Treffer zu zählen. | Filtert offensichtlichen Müll am Adapter-Rand. |
| `LINK_QUALITY_THRESHOLD` | Ein dienstübergreifender Link muss mindestens diesen Wert erreichen, um in die Antwort aufgenommen zu werden. | Hält verrauschte Fuzzy-Treffer von der Anzeige fern. |
| `AUTO_SELECT_THRESHOLD` | Eine Textsuche löst direkt auf, wenn der vorderste Vorschlag mindestens diesen Wert erreicht. | Spart Folgeanfragen bei klaren Fällen. |
| `CANDIDATE_MIN_CONFIDENCE` | Ein Vorschlag muss mindestens diesen Wert erreichen, um in der Vorschlagsliste zu erscheinen. | Hält hoffnungslose Optionen aus der Disambiguierung. |
| `MAX_CANDIDATES` | Maximale Länge der Vorschlagsliste. | Begrenzt die Antwortgröße. |
| `SEARCH_FALLBACK_CONFIDENCE` | Konfidenz, die synthetischen Such-Notlösungen zugewiesen wird. | Sortiert solche Links ans Ende der Liste. |

Der Artist-Resolver verwendet eine eigene Konfidenzformel, weil er auf Namensähnlichkeit prüft und nicht, wie der Track-Resolver, auf Identität von ISRC oder Text. Der Album-Resolver übernimmt die vom

Adapter gemeldete Konfidenz, wenn die Strategie eine Textsuche ist; für Treffer per UPC oder per ISRC-Ableitung sind die Werte fest gesetzt.

## 11 Hinweise für den Betrieb

Einige Punkte, die bei Arbeiten an dieser Schicht hilfreich sind:

**Adapter-Zeitschranke.** Die dienstübergreifende Suche lässt jeden Adapter-Aufruf gegen eine eigene Zeitschranke laufen. Ohne sie würde ein einzelner blockierender Adapter die ganze Antwort aufhalten, bis der allgemeine HTTP-Timeout greift, was deutlich länger dauert.

**Cover-Notlösungen.** Manche Dienste liefern in ihrer Track-API kein Cover. Fehlt es bei der Quelle, ist aber ein ISRC bekannt, holt der Resolver ein Cover aus den Treffern der übrigen Dienste. Spotify und Apple Music liefern in dieser Hinsicht gleichbleibend gute Qualität und stehen deshalb am Anfang der Reihenfolge.

**User-Agent für Open-Graph-Auslesung.** Single-Page-Anwendungen liefern je nach User-Agent unterschiedliche Inhalte aus und stellen Open-Graph-Metaangaben oft nur für bekannte Crawler bereit. Der Open-Graph-Pfad gibt sich genau aus diesem Grund als ein solcher Crawler aus, um die servergerenderte Variante zu erhalten.

**Spotify zuerst bei der Textsuche.** Die Adapter werden in einer festen Reihenfolge durchgegangen. Spotify steht ganz vorn, sobald sein API-Schlüssel gesetzt ist, weil seine Relevanzwertung die zuverlässigste Grundlage bietet. Alle weiteren Adapter dienen als Auffangnetz.

**Statisch-dynamische Trennung.** Kanonische Einträge sind seit der Cache-Migration zeitlich unbegrenzt gültig. Was veralten kann, lebt in eigenen Hilfstabellen mit eigenen Aktualitätsregeln. Diese Trennung hält den schnellen Pfad bei einem Cache-Treffer einfach.

**Schwellwerte an einer Stelle.** Alle Konfidenz-Werte liegen in einem einzigen Modul. Eine Verhaltensänderung ist deshalb eine Zahlenänderung in einer Datei statt einer Suche nach verstreuten Konstanten.

## 12 Eine neue Verzweigung oder Strategie hinzufügen

Wer dem Resolver eine neue Eingabeform oder eine neue Strategie hinzufügen möchte, geht dabei in vier Schritten vor:

1. Die zugehörige Diagrammquelle in `docs/resolve-flow/diagrams/` aktualisieren.
2. Den Prosatext in der Datei `resolve-flow.tex` entsprechend anpassen.
3. `make docs` im Wurzelverzeichnis des Repositorys ausführen.

4. Die geänderte Diagrammquelle, die regenerierten Diagramm-PDFs und die neue `resolve-flow.pdf` zusammen mit der zugehörigen Code-Änderung committen.

Die LaTeX-Datei ist die einzige Quelle für den Prosatext, die Diagramm-Dateien sind die einzige Quelle für die Diagramme. Beide zusammen ergeben über XeLaTeX die fertige PDF.

## Glossar

[1]ISRC, der International Standard Recording Code, ist eine zwölfstellige alphanumerische Kennung, die einer einzelnen Aufnahme zugewiesen wird. Da verschiedene Streaming-Dienste denselben ISRC für dieselbe Aufnahme verwenden, ist er die zuverlässigste dienstübergreifende Track-Kennung, über die der Resolver verfügt.

[2]UPC, der Universal Product Code, ist ein zwölfstelliger Identifier in Form eines Strichcodes, der ein Album-Release auf Industrieebene benennt. Beim Resolver übernimmt der UPC für Alben dieselbe Rolle wie der ISRC für einzelne Aufnahmen.

[3]MBID, der MusicBrainz Identifier, ist eine UUID, die von der MusicBrainz-Datenbank vergeben wird. Sie identifiziert eine Aufnahme, ein Werk, einen Interpreten oder ein Release auf einer feineren Ebene als der ISRC und ist pro Datenbank-Entität eindeutig.

[4]ISWC, der International Standard Musical Work Code, ist eine elfstellige Kennung, die ein musikalisches Werk benennt, nicht eine konkrete Aufnahme davon. Mehrere Aufnahmen desselben Werks teilen sich einen ISWC, auch wenn sie unterschiedliche ISRCs haben.